

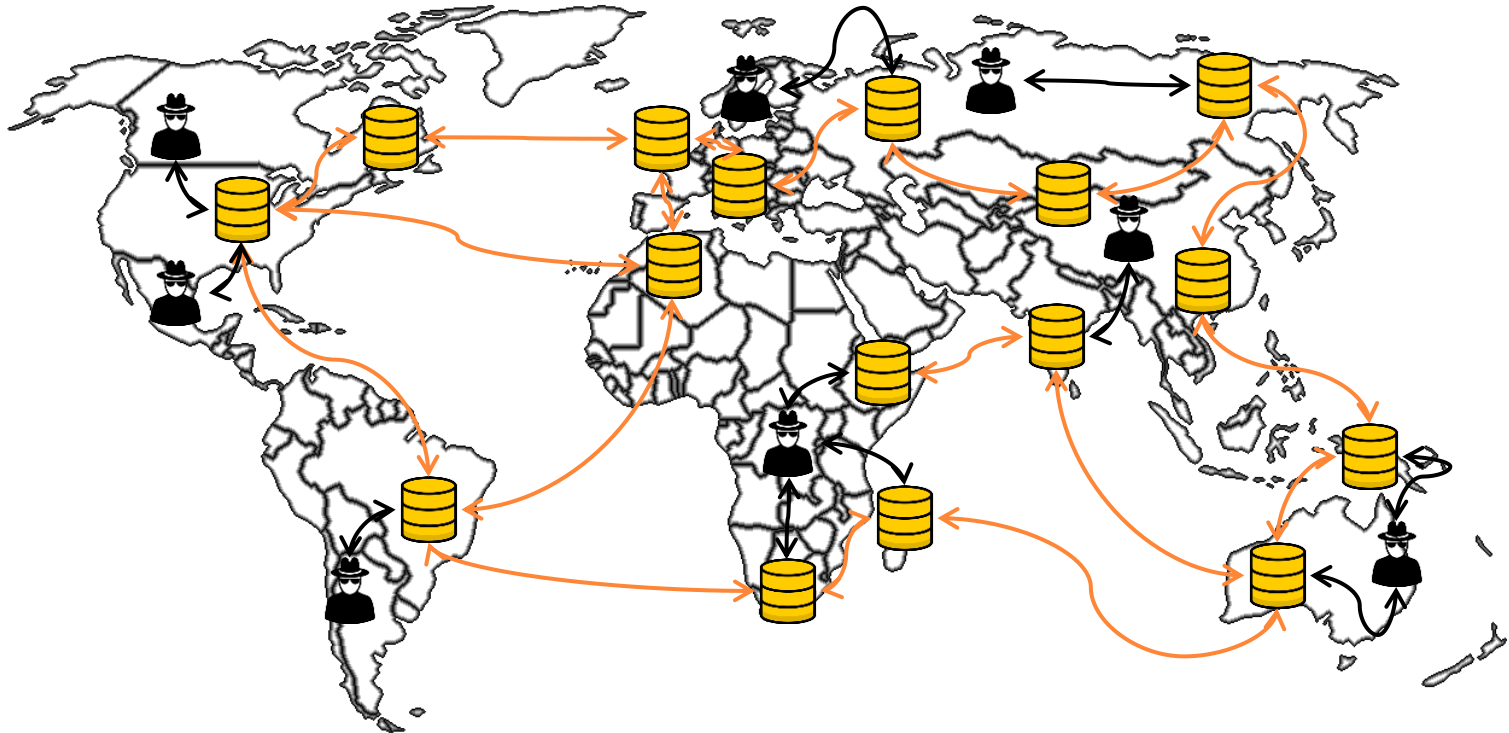


DISTRIBUTED SYSTEMS - PROJECT 2026:

A QUORUM-BASED TOTAL ORDER BROADCAST PROTOCOL FOR DISTRIBUTED INTELLIGENCE DATABASES

INTELLIGENCE SERVICES

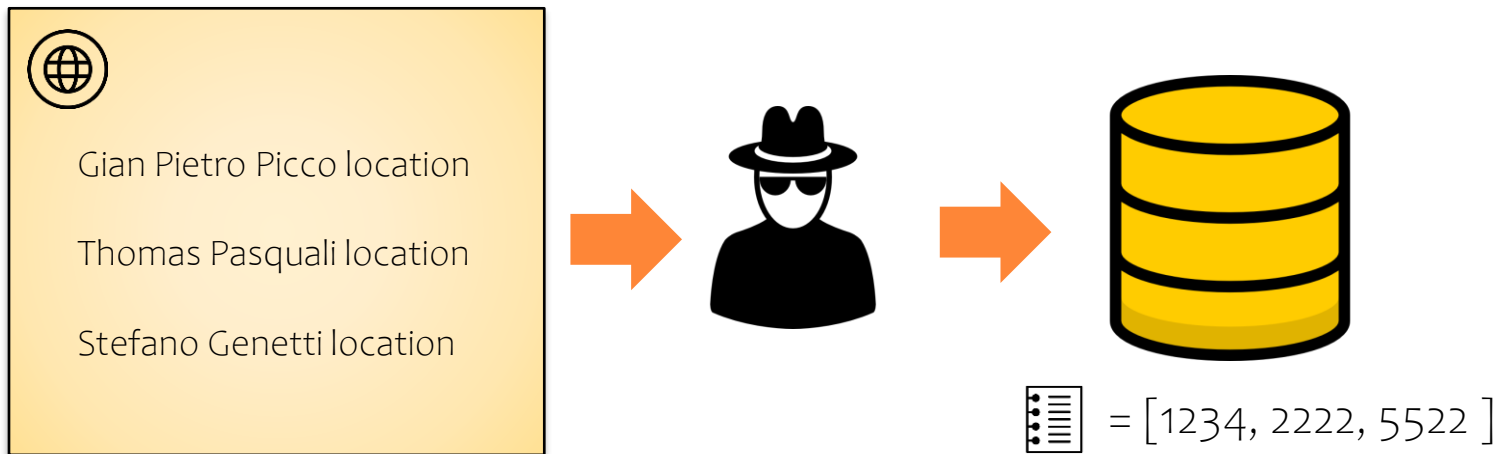
The intelligence services use a list of geographical positions of people of special interest. The list is stored in a distributed storage system geographically distributed worldwide.



Your task is to design a distributed protocol for coordinating the distributed system, ensuring consistent management of the shared list.

INTELLIGENCE SERVICES

Each position is serialized as an integer value. The positions are therefore stored in an array of integers, with each index corresponding to a specific person of special interest. The size of the list is fixed.



 List of geographical positions of persons of special interest, encoded as a list of integers and stored within the distributed system.

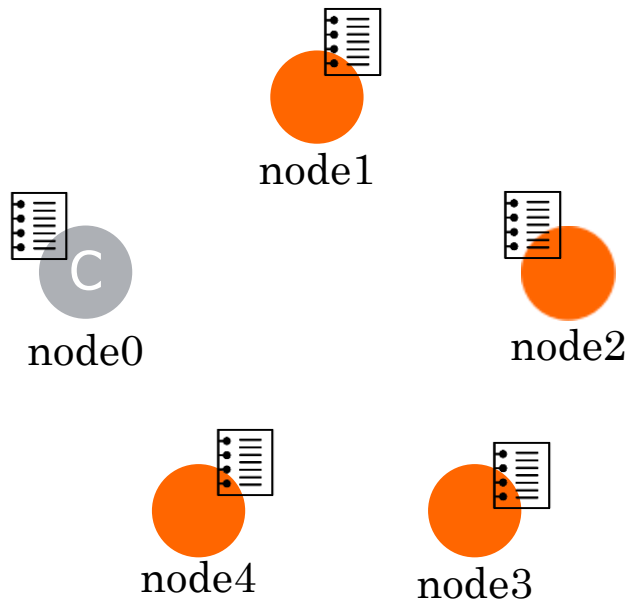
TOTAL ORDER BROADCAST

- Reliable broadcast of message m satisfies:
 - **Validity** - if the sender is correct, then it will eventually deliver m .
 - **Integrity** - a process delivers m at most once, and only if it was previously sent.
 - **Uniform Agreement** - if a process delivers a message m , then all correct processes deliver m .
- **+Total order**: if a correct process delivers a message m before a message m' , then all correct processes deliver m before m' .

In this project, you will implement a protocol for Total Order Broadcast (or Atomic Broadcast) that relies on a coordinator and a quorum of nodes.

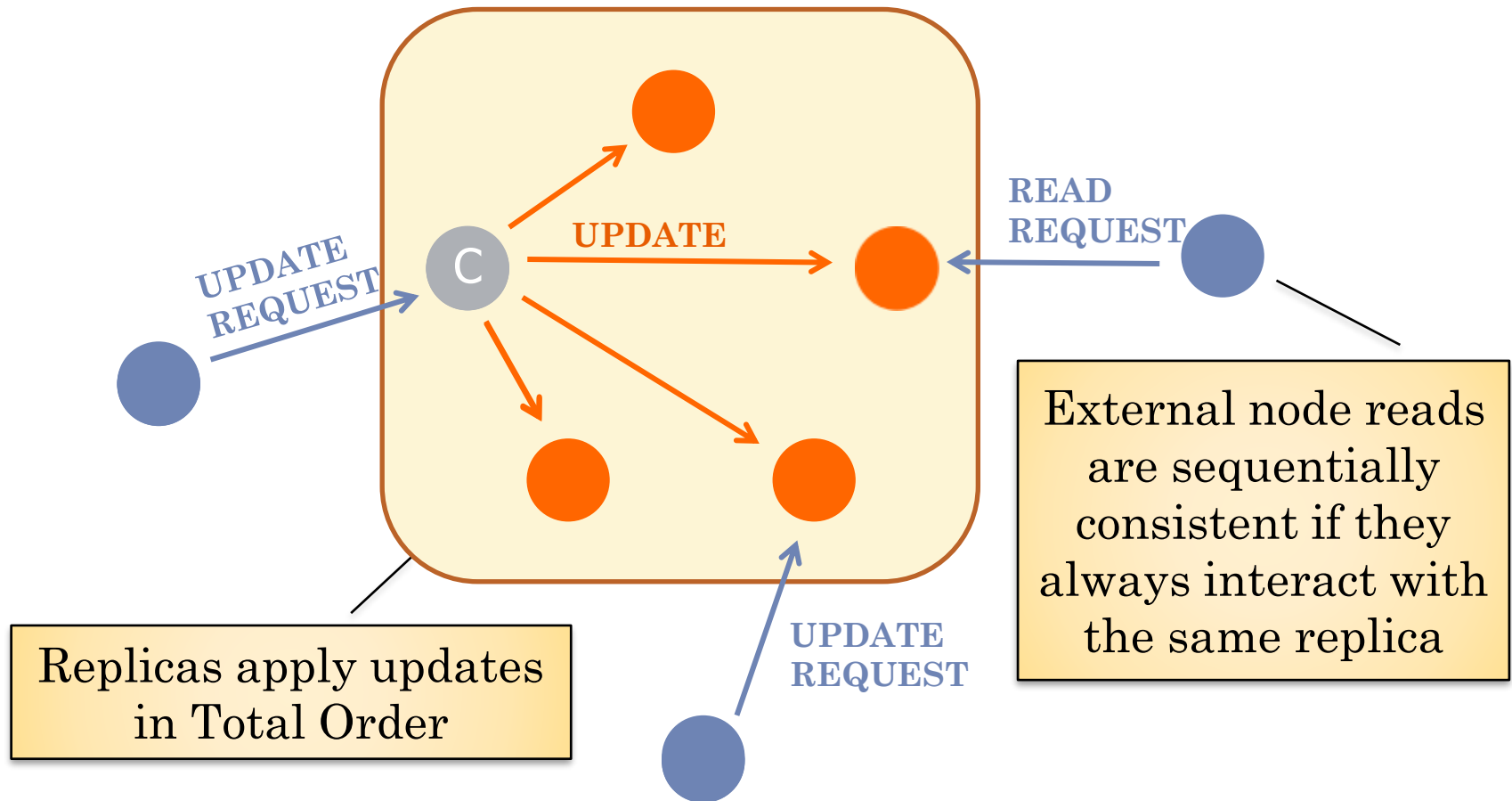
CENTRALIZED BROADCAST WITH QUORUM

- It allows to reliably update the state of nodes in the distributed system by means of a **coordinator**.
- Assumption: a **strict majority of nodes** is always available.



In this example there are five replicas, hence the quorum Q is 3.

SYSTEM OVERVIEW



Clients may issue read and write requests to any replica.

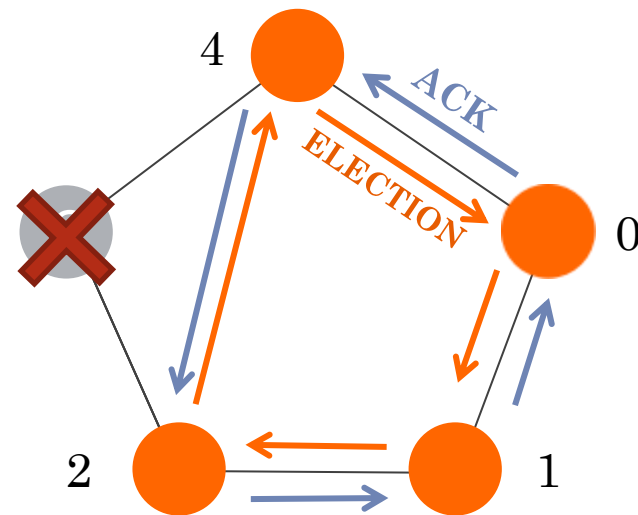
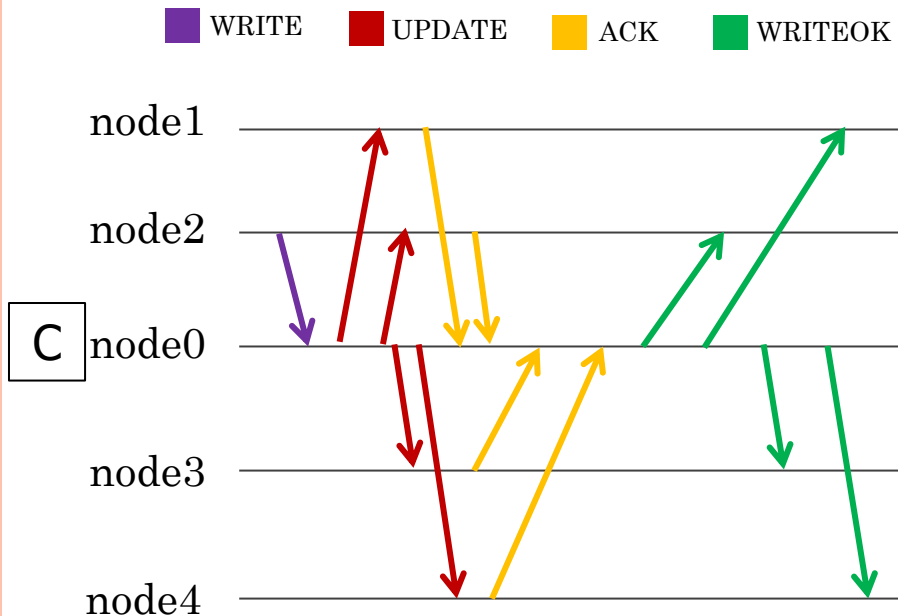
Examples: $\text{read}(\text{idx}) \rightarrow \mathcal{P}[\text{idx}]$

$\text{write}(\text{idx}, \text{val}) \rightarrow \mathcal{P}[\text{idx}] = \text{val}$

$\mathcal{P}$ is the list of integer-encoded locations

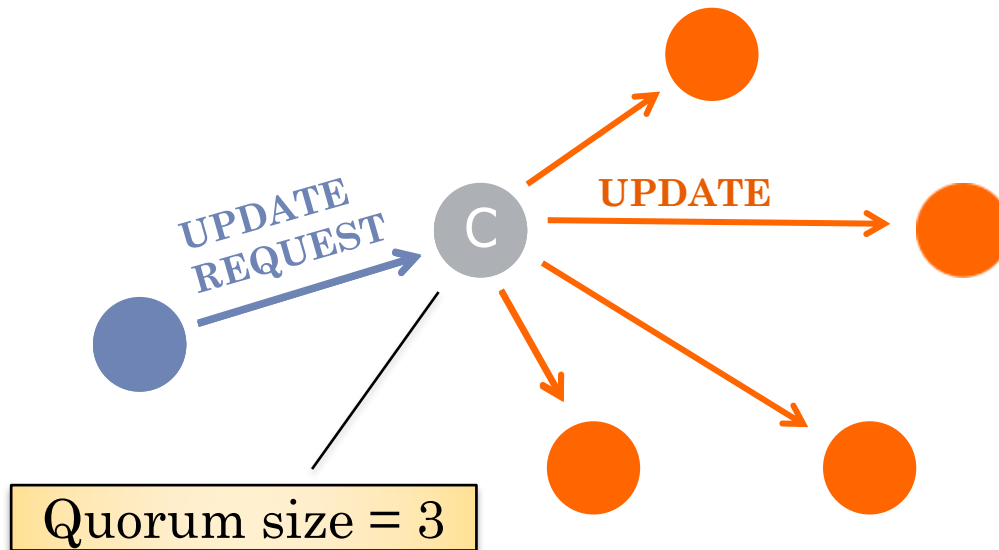
SYSTEM OVERVIEW

- The system has two main components:
 - Two-phase broadcast** (with a quorum)
 - Coordinator election** (on a ring)



THE BROADCAST PROTOCOL

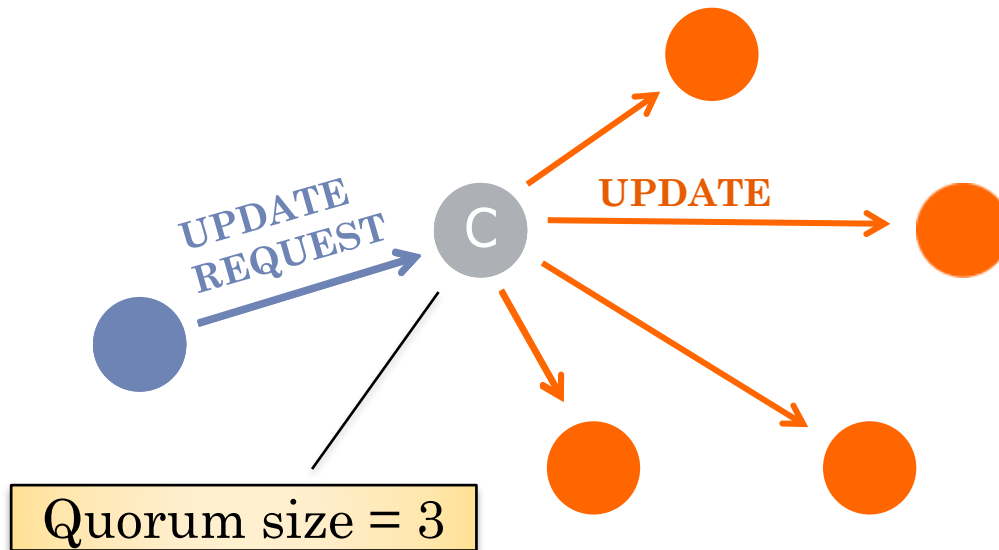
- Two-phase broadcast: prompted by an external request, the coordinator sends the update to all nodes, and waits for a majority to ACK.



- Each update the coordinator sends is identified by a pair $\langle e, i \rangle$. e is the epoch that corresponds to a coordinator, i is a sequence number.

THE BROADCAST PROTOCOL

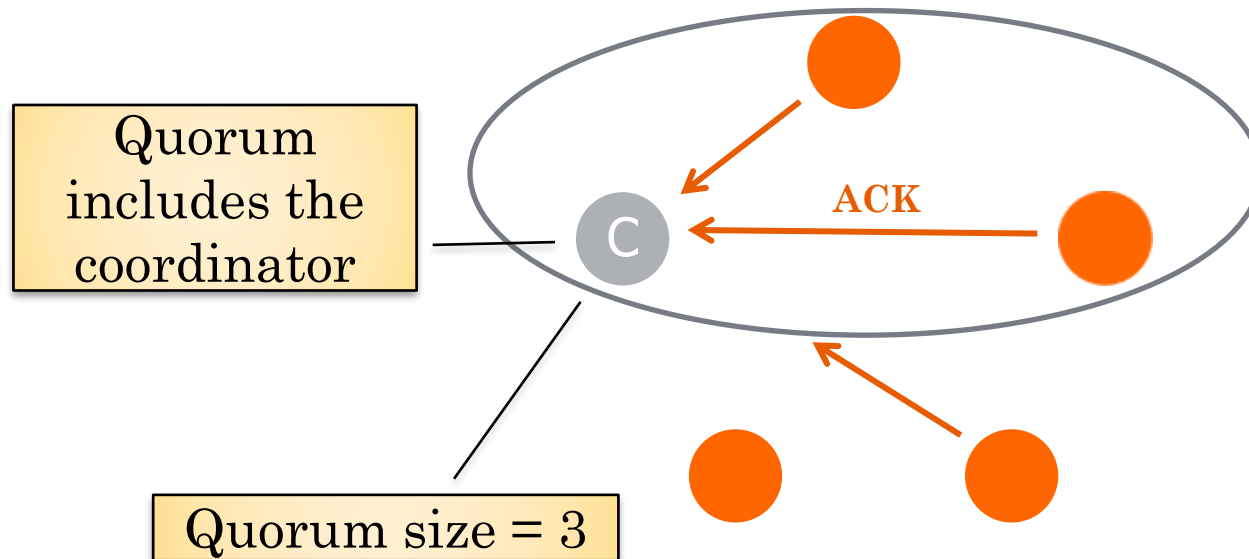
- Two-phase broadcast: prompted by an external request, the coordinator sends the update to all nodes, and waits for a majority to ACK.



e is monotonically increasing, while i restarts from 0 at every epoch (coordinator) change.
 $\langle e, i \rangle$ precedes $\langle e', i' \rangle$ iff $e < e'$ or ($e = e'$ and $i < i'$)

THE BROADCAST PROTOCOL

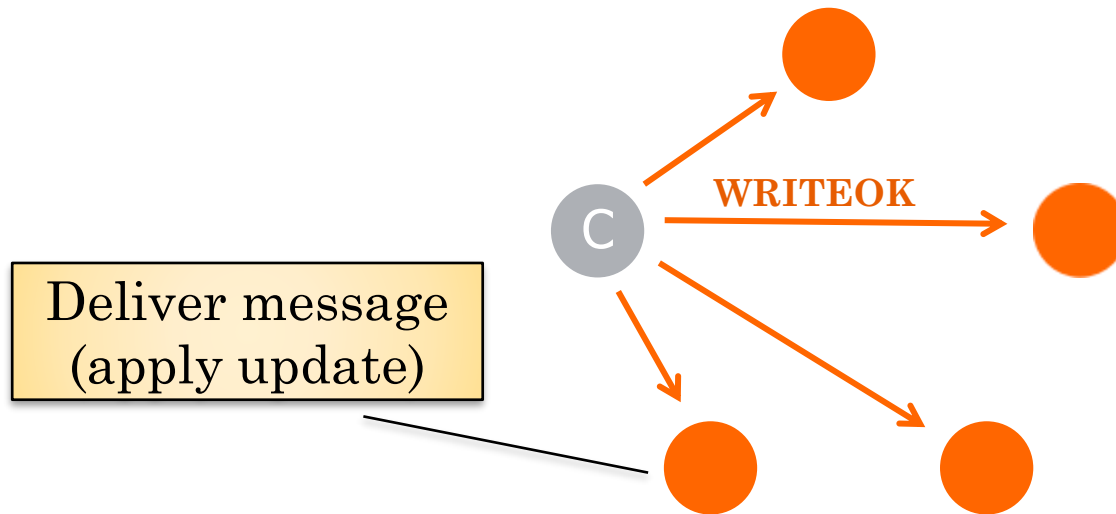
- Two-phase broadcast: prompted by an external request, the coordinator sends the update to all nodes, and waits for a majority to ACK.



- In this example, as soon as (any) 3 nodes **ACK**, the coordinator broadcasts another message to confirm the update.

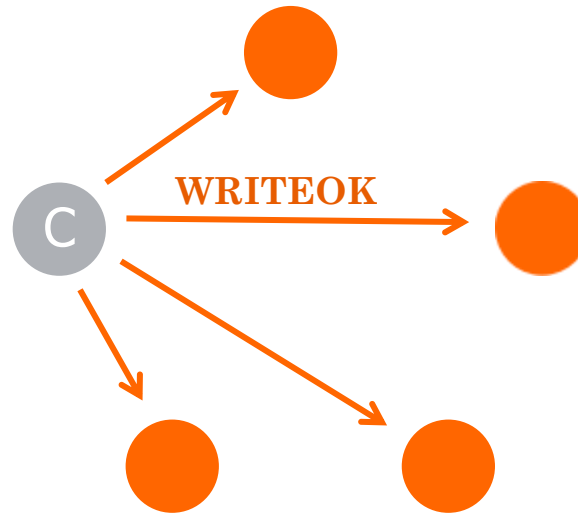
THE BROADCAST PROTOCOL

- Two-phase broadcast: prompted by an external request, the coordinator sends the update to all nodes, and waits for a majority to ACK.



- Finally, the coordinator sends the **WRITEOK** message to confirm the update. Replicas can now apply it.

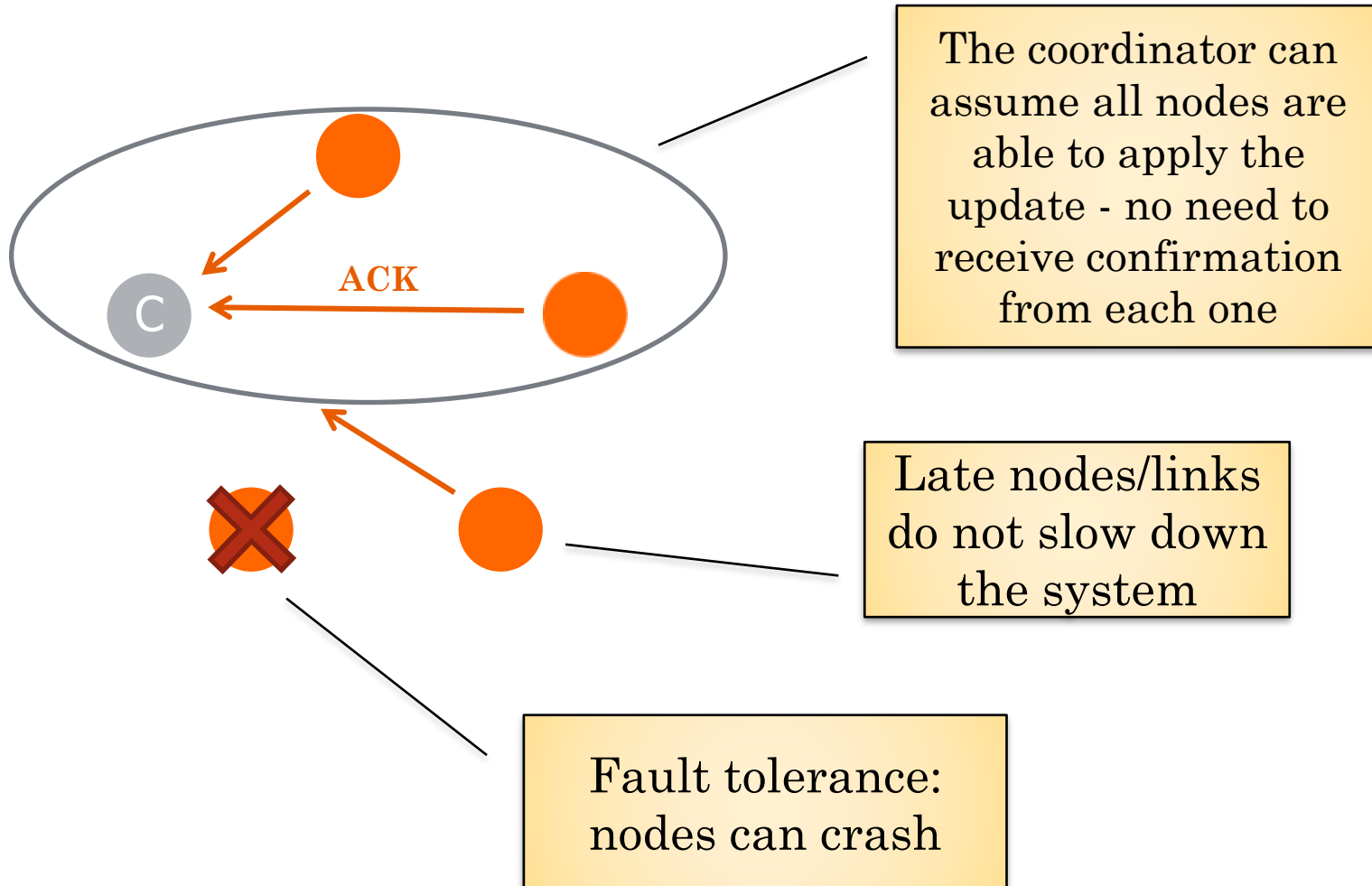
THE BROADCAST PROTOCOL



- We assume channels are reliable and FIFO.
→ eventually, all operational nodes deliver the updates in the same order as the coordinator.

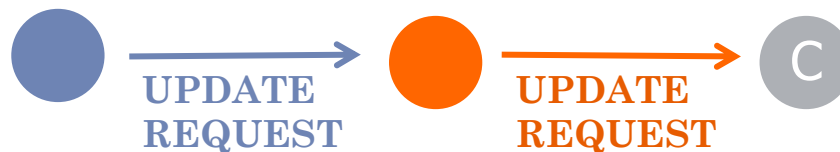
THE BROADCAST PROTOCOL

- Why ACKs from a quorum?



UPDATE AND READ REQUESTS

- In this project, there are two types of requests: update (**write**) and **read**. They can be sent by external nodes to any replica in the system.
- A **read REQUEST** message is managed by the replica receiving it: it replies immediately with its local list of integer values.
- An **update REQUEST** message will be forwarded to the coordinator, which will initiate the two-phase broadcast.

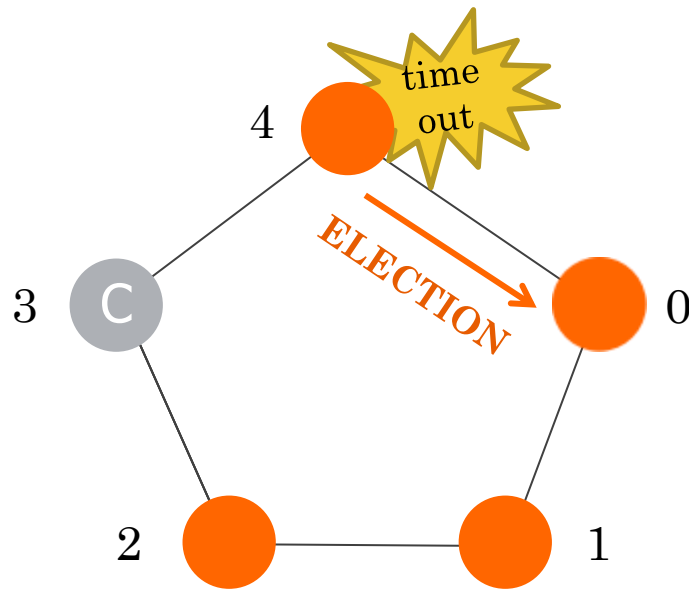


NODE FAILURE

- Any node, including the coordinator, can crash at any time. Replicas do not recover.
- Nodes detect a crashed coordinator by means of timeouts.
 - If the **WRITEOK** message is not received on time after the **UPDATE**.
 - If the **UPDATE** message is not received on time after forwarding an **update REQUEST** to the coordinator.
 - If the coordinator was not heard for too long. The coordinator periodically sends a **HEARTBEAT**.
- If the coordinator fails, the remaining nodes need to **elect a new one**.

COORDINATOR ELECTION

- We will use a leader election protocol based on a ring topology, defined by unique IDs of replicas.

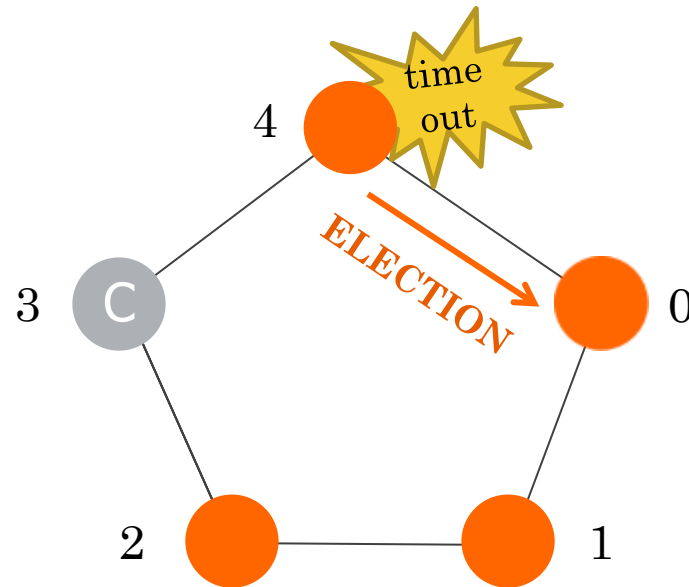


Please note that replica IDs and the integers stored within each replica refer to different things. In this context, we are referring specifically to the node's ID.

- When a replica detects a crashed coordinator, it begins the election by sending the **ELECTION** message to the next participant in the ring.

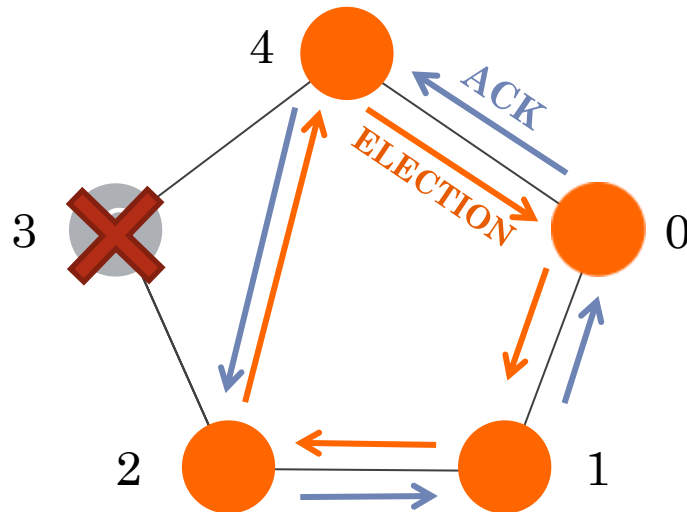
COORDINATOR ELECTION: MOST RECENT UPDATE

- The goal of this election protocol is to find the best candidate to take the role of coordinator. The best candidate is the replica that knows the most recent update in the system.
- To achieve this, the information about the last update (associated to the node ID) should be appended to the **ELECTION** message. Eventually, nodes will be able to compare all candidates in **ELECTION**, and decide the winner. If there are ties, use the node ID to disambiguate.



COORDINATOR ELECTION

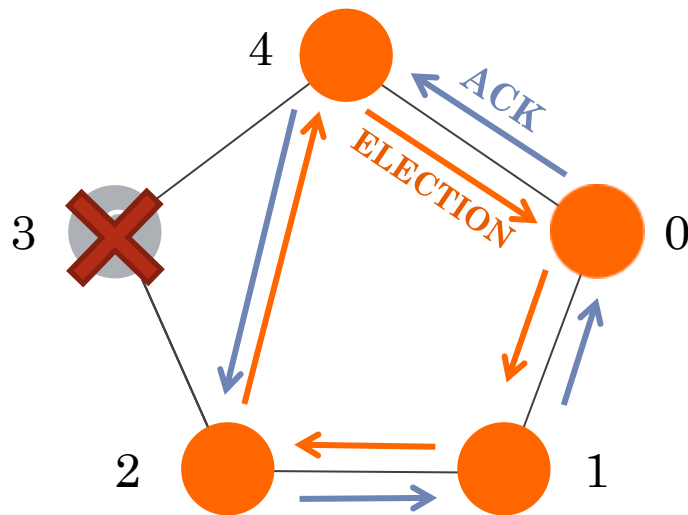
- In the ring algorithm, the **ELECTION** message is repropagated until a coordinator candidate is found.



- Upon the reception of **ELECTION**, it will add its information about the most recent update, forward to the next node and then **ACK** the sender.

COORDINATOR ELECTION

- Eventually, at some point, a node will receive an **ELECTION** message and recognize that the election process has been completed.

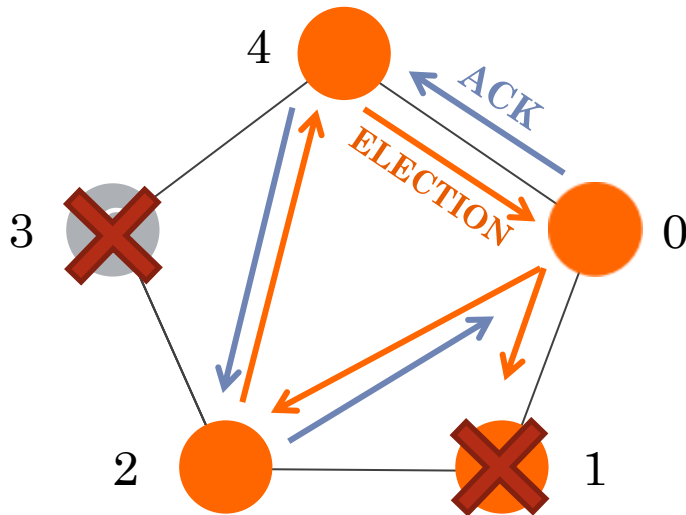


- If it is the best candidate, it declares to be the winner and sends a **SYNCHRONIZATION** message in broadcast to provide other nodes with all missing updates.
- If it is not the best candidate, it will forward the message (at some point, it will reach the winner).

Hint 1: make sure that the most recent update does not change while a node is participating in the election!

COORDINATOR ELECTION: CRASH IN THE RING

- Nodes may crash even during leader election.



- Forwarders timeout if they do not receive an **ACK**. In that case, they will “skip” the crashed node and forward to the next node in the ring.
- Note that, if we assume a majority of nodes remains available, at least one of them must hold the most recent update.

COORDINATOR ELECTION: TERMINATION

- There are corner cases in which the election described so far may not terminate.
- This can happen if two consecutive nodes crash in a way that cannot be detected by the preceding correct sender. In that case, other nodes may wait forever.
- If the best candidate crashes, make sure the **ELECTION** message does not loop endlessly!

Hint 2: nodes should expect the election to end after some time has passed, otherwise...

UNIFORM AGREEMENT

- During an election, you may encounter an incomplete update (no WRITEOK). Since the coordinator crashed, we cannot tell whether or not it applied the update.
- Uniform agreement requires that if any (not necessarily correct) process delivers, then all correct processes do as well.
- Otherwise, some external clients may read a value that is not in the history of any correct node.

Hint 3: if an update was not completed, the new coordinator should take care of it before beginning with its epoch.

ASSUMPTIONS

- Reliable and FIFO channels.
- The mechanisms for failure detection do not give false positives. Use reasonable values for timeouts.
- A majority of nodes does not crash.
- Nodes that crash do not recover.

IMPLEMENTATION SUGGESTIONS

- Start by implementing the two-phase broadcast protocol without crashes. Manage read requests from external clients (that will timeout if no reply is received) and update requests (in this case, clients do not need to wait for the outcome).
- Create a class for the $\langle e, i \rangle$ pair, so that you can use it as a key in a map of updates. A dedicated data structure for the history of updates may come in handy.
- You will need to manage many timeouts. Create a method to schedule them. Prevent a timeout message to be added to the queue calling `.cancel()` on the `Cancellable` returned by the scheduler. Note that if you schedule a timeout twice using the same variable without cancelling first, `.cancel()` cannot stop the first one scheduled.

IMPLEMENTATION SUGGESTIONS (2)

- Create different Receive behaviors for the *crashed* and *election* states. This will help you control how messages are handled (or ignored) in every state.
- Be careful not to deliver duplicated updates!
- You will need to show that your implementation is correct in the presence of faults. You should be able to make nodes crash in specific situations.

PROJECT REPORT

- Structure of the project
- Example of questions that your report is expected to address.
 - How do you store updates?
 - How do you manage timeouts?
 - How do you control crashes?
 - What operations does a new coordinator perform?
 - How do you ensure election does not block forever?
 - ...

CODEBASE AND AUTOMATED TESTS

- You will be given a codebase to start developing your project. The codebase is available at:
<https://github.com/StefanoGenettiUniTN/ds-student-project-2026>
- It is **mandatory** to use it, read the documentation, and adhere to the guidelines.
- The codebase will provide some basic automated tests. Submissions that do not pass **ALL** tests will not be considered. Not adhering to the codebase requirements will make tests fail.